TTK4190 Guidance, Navigation and Control of Marine Craft, Aircraft and Drones

Assignment 2 part 3

Written Fall 2026

Mellingsæter, Odin Sandlie
odinsm@stud.ntnu.no

Skaset-Haarr, Mathilde
mathskas@stud.ntnu.no

November 13, 2026

Problem 1 - LOS Guidance Law for Straight-Line Path Following

Problem 1a

The following LOS guidance system was implemented:

```
%LOS Guidance
function chi_d = LOS_guidance(pi_h, y_ep)
L_pp = 161;
look_ahead = 6*L_pp;  %tuning parameter

Kp = 1 / look_ahead;


chi_d = pi_h - atan( y_ep* Kp);

end
```

The only tunable parameter in this function is $K_P = \frac{1}{\Delta}$. Choosing a small look ahead distance Δ would result in a large proportional gain and more aggressive stearin [1].

The straight line segments in the planned path differ in sharpness of the angles between them, with the angle between way-point 2 and 3 being the sharpest. The look ahead distance was chosen small enough to accommodate sharpest change in angle between waypoints, while still preventing the system from becoming to aggressive. After experimenting with different values, the group ended up choosing $\Delta = 6$. This allowed the craft to keep up with way-point changes fairly well, while not producing unnecessary large overshoots.

Problem 1b

The following changes was made to the main code from the previous assignment.

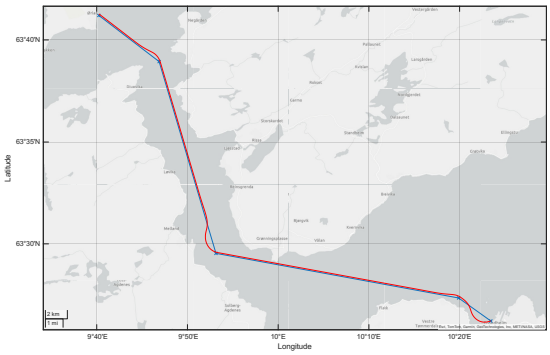
```
%----- Guidance law -----------
[xk1,yk1,xk,yk,last] = WP_selector(x(4),x(5));
[pi_h, y_ep]  = crossTrackError(xk1,yk1,xk,yk,x(4),x(5));
chi_d = LOS_guidance(pi_h, y_ep);
psi_ref = chi_d;
```

After implementing the changes shown above, the ship started spinning in circles around the first way-point. This was resolved after switching from a PID regulator to a PD regulator, as suggested by the hint in the assignment description[2].

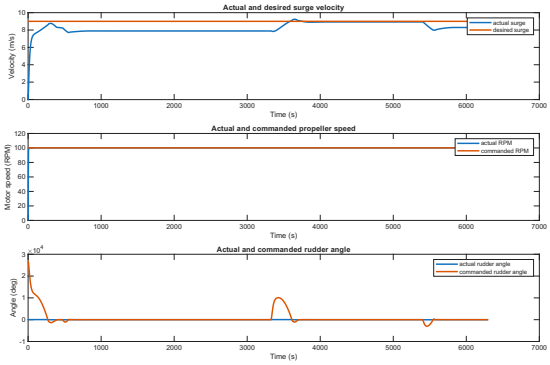
Problem 1c

The system response after introducing LOS guidance can be seen in figure 1. The craft is able to follow the path fairly well, with some overshooting when changing the heading angle to accommodate a new way-point. This is expected to some extent, as a vessel of this size cannot change its heading instantaneously.

In the `WP_selector`, the radius of acceptance is set to two times the length of the ship. This means that the ship will change its next desired way point when it is within this distance from of its current waypont. Since the vessel will need time to adjust its heading, it may overshoot slightly before settling onto the next path segment.



(a) Path plot



(b) Actual and desired Surge velocity, propeller speed and rudder angle

Figure 1: Simulation results after introducing LOS guidance

To examine the effect of the current, an additional simulation was ran without the effect of current, but otherwise with the same parameters as in Figure 1.

Comparing the result in figure 1 and 2, it appears that the current does affect the results. In figure 2a, we observe that the overshoot after each way point is slightly smaller than in figure 1a. With no current, the ship is also able to recover from the overshoot better. After recovering from the overshoot, the ship follows the line segment almost perfectly. With the current, the ship will consistently follow the line segment with a small offset.

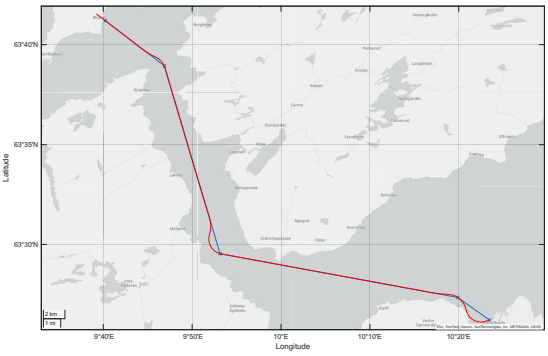
This is expected, as the LOS guidance law implemented only contains a proportional gain K_P which does not account for constant disturbances like ocean current. Introducing a integral gain might allow for straight path-following while being influenced by the ocean current.

Overall the path-following behavior is as expected.

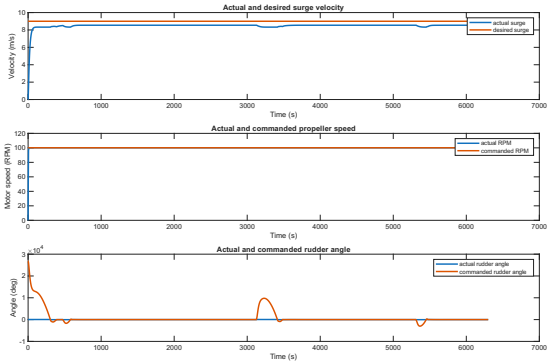
Problem 2 - Crab Angle Compensation and Integral LOS

Problem 2a

The simulation results after turning the current off can be seen in figure 3. Based on the simulation result, I would not claim that any crab angle compensation is necessary. As can be observed in figure 3c, the crab angle stays around 0, only spiking when the ship is turning. When looking at figure 3a and 3b, both the course and heading are able to follow the respective references fairly well with close to no deviation when the ship is not actively turning.



(a) Path plot after removing the effect from current



(b) Actual and desired Surge velocity, propeller speed and rudder angle

Figure 2: Simulation results without current

Problem 2b

The simulation results after turning the current back on can be seen in figure 4. When the the current is reintroduced the heading dynamics does not change much, as can be observed in figure 4b. However, a noticeable change occurs in both the crab angle and the course. In figure 4c, we observe that the crab angle now stays consistently above zero, also when the ship is not turning. Since the course χ is defined as[1]:

$$\chi = \psi + \beta_c. \tag{1}$$

and no compensation is made for the crab angle being non-zero, this introduces a persistent tracking error in the course, also when the ship is supposed to move straight forward. This can clearly be observed in figure 4a.

Based on the observations mentioned above, I would claim that crab angle compensation is necessary in this case.

Problem 2c

To compensate for the none-zero crab angle, the heading reference ψ_{ref} was adjusted by subtracting the crab angle at each step.

```
%crab angle compansation
beta_c_i = atan2(x(2), x(1));
psi_ref = psi_ref-beta_c_i;
```

The results of the simulation with both currents and crab angle compensation can be seen in figure 5.

We now observe that with crab angle compensation, the simulation results becomes similar to the results obtained without the current(figure 3). The persistent tracking error observed in 4a has effectively been eliminated, as can be seen in figure 5a.

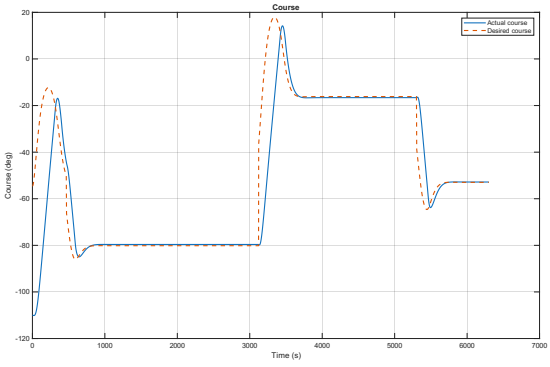
The results listed above indicates that the crab angle compensation successfully counteract some of the effects the current has on the ships ability to follow the desired heading.

Problem 2d

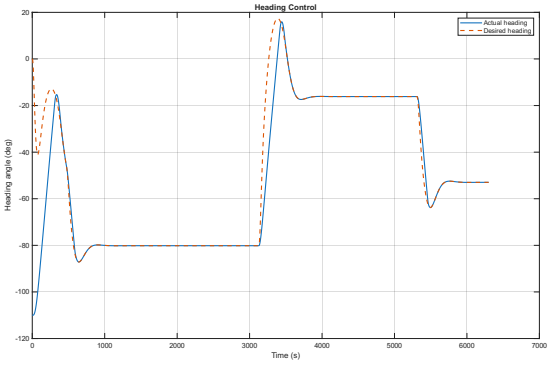
The ILOS guidance law implemented can be found below:

```
%ILOS Guidance
function chi_d = ILOS_guidance(pi_h, y_ep, h)

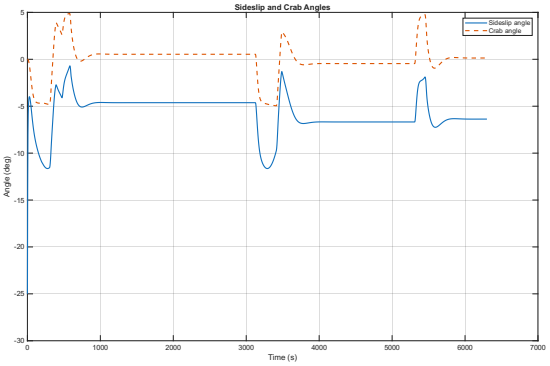
    persistent y_int
```



(a) Actual and desired Course



(b) Actual and desired heading



(c) Sideslip and crab angles

Figure 3: Simulation results without current

```
    if isempty(y_int)
        y_int = 0;
    end

    L_pp = 161;
    look_ahead = 6*L_pp;  %tuning parameter
    kappa = 0.5;          %tuning parameter

    Kp = 1 / look_ahead;
    Ki = kappa*Kp;

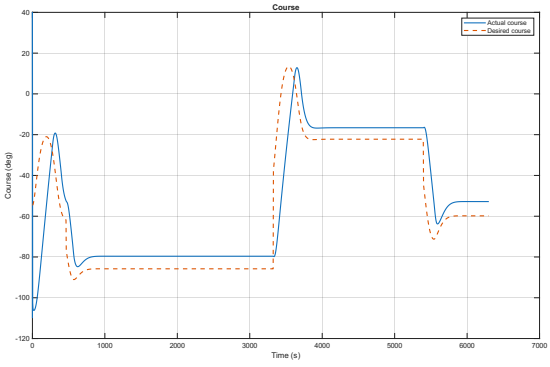
    chi_d = pi_h - atan( y_ep* Kp + Ki*y_int);

    y_int_dot = (look_ahead*y_ep) / (look_ahead^2 + (y_ep + kappa*
        y_int)^2);
    y_int = y_int + y_int_dot*h;

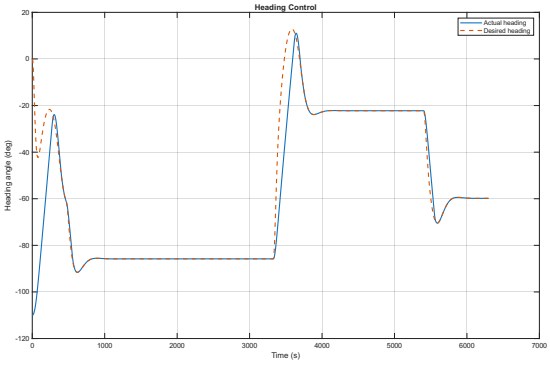
end
```

The tuning parameter for the integral κ was chosen as a compromise between fast compensation and preventing windup and overshoots. Choosing κ larger than 0.5 introduced oscillations into the system. Although Anti-windup techniques could have been utilized to prevent this, a smaller κ proved to have a satisfying performance while keeping the oscillations to a minimum.

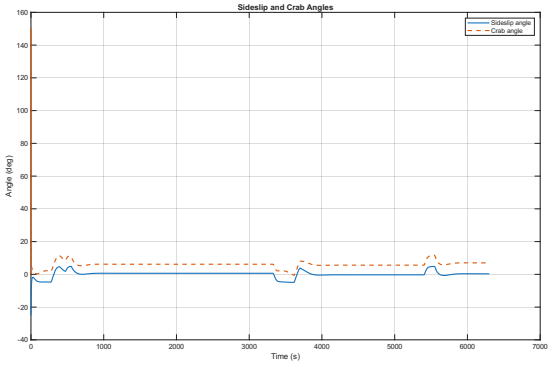
Figure 6 show the simulation results when using the ILOS guidance law. The integral effect in ILOS allows the ship to follow the course despite the influence of current and non-zero crab angles[1]. In figure 6a the tracking error observed in task 2b is now being compensated for by the integral action, resulting in nearly zero tracking error on the straight line segments. Removing the tracking



(a) Actual and desired Course



(b) Actual and desired heading



(c) Sideslip and crab angles

Figure 4: Simulation results with current

error results in slightly better path-following, as can be seen in figure 6d. The ship is now able to converge on the exact path-line some time after performing the turn towards the next waypoint.

A pro of using using ILOS, is that there is no need to know the exact crab angle. Since the crab angle is dependent on the environment, it is not always well defined or straight-forward to determine. One example is at very low speeds[3]. Since the exact crab angle is not always known, it is not always possible to use it for direct compensation.

A con of using ILOS instead of direct crab angle compensation, is the risk of overshoot and integrator wind-up. Additionally, since the integrator will need time to accumulate the error, the convergence might be relatively slow compared to direct compensation.

Problem 3 - Kalman Filter Design

Problem 3a

The first order Nomoto model derived in assignment 2 part 2 is:

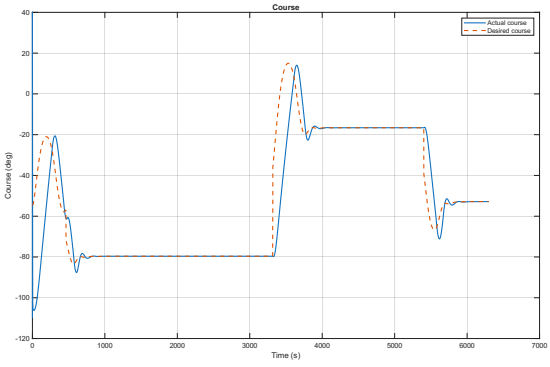
$$\dot{r} = -\frac{1}{T}r + \frac{K}{T}\delta \tag{2}$$

Subtracting the rudder bias gives:

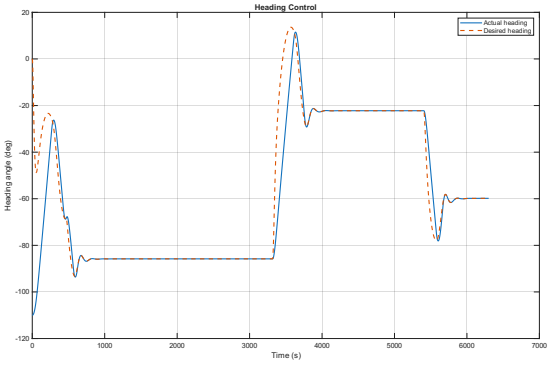
$$\dot{r} = -\frac{1}{T}r + \frac{K}{T}(\delta - b) \tag{3}$$

The rudder bias is modeled as a random-walk process by white noise. $\dot{\psi}$ we find from kinematic relations. The last two equations become:

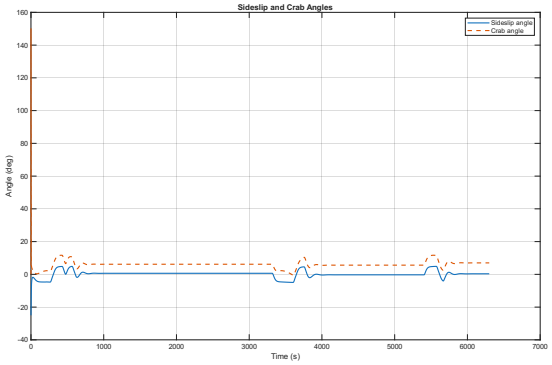
$$\begin{aligned} \dot{\psi} &= r \\ \dot{b} &= w_b \end{aligned} \tag{4}$$



(a) Actual and desired Course



(b) Actual and desired heading



(c) Sideslip and crab angles

Figure 5: Simulation results with current and crab angle compensation

Defining the state vector as

$$\mathbf{x} = \begin{bmatrix} \psi & r & b \end{bmatrix}^\top \tag{5}$$

the continuous-time state-space model is

$$\dot{\mathbf{x}} = A\mathbf{x} + B\delta + Ew_b \tag{6}$$

with

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & -\frac{1}{T} & -\frac{K}{T} \\ 0 & 0 & 0 \end{bmatrix}, \qquad B = \begin{bmatrix} 0 \\ \frac{K}{T} \\ 0 \end{bmatrix}, \qquad E = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}. \tag{7}$$

Since yaw is the only measured state, the measurement equation is

$$y = C\mathbf{x} + D\delta + v \tag{8}$$

where v represents gyroscope measurement noise. Thus:

$$C = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}, \qquad D = 0. \tag{9}$$

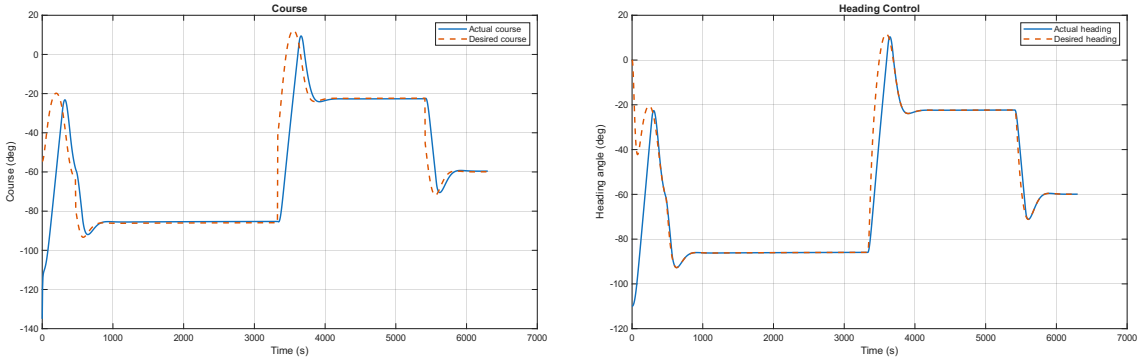
Problem 3b

The continuos-time system derived in 3a is discretized using the first-order forward Euler approximation

$$\dot{\mathbf{x}} \approx \frac{\mathbf{x}_{k+1} - \mathbf{x}_k}{h} \tag{10}$$

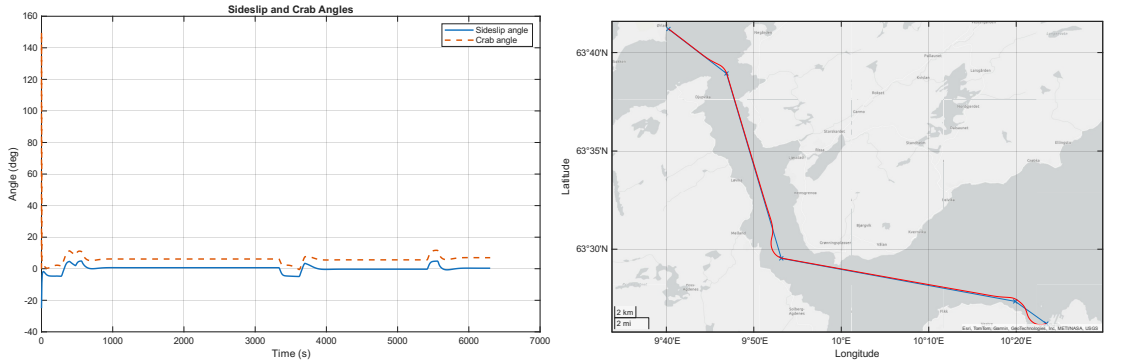
Substituting into the continuous-time state-space model and rearrenging gives

$$\mathbf{x}_{k+1} = (I + hA)\mathbf{x}_k + hB\delta_k + w_k \tag{11}$$



(a) Actual and desired Course

(b) Actual and desired heading



(c) Sideslip and crab angles

(d) Path plot

Figure 6: Simulation results after introducing ILOS

where w_k is the discrete process noise. Therefor,

$$A_d = I + hA, \qquad B_d = hB \tag{12}$$

This gives

$$A_d = \begin{bmatrix} 1 & h & 0 \\ 0 & 1 - \frac{h}{T} & -\frac{hK}{T} \\ 0 & 0 & 1 \end{bmatrix} \tag{13}$$

and

$$B_d = \begin{bmatrix} 0 \\ \frac{hK}{T} \\ 0 \end{bmatrix} \tag{14}$$

The measurement matrices remain unchanged:

$$C_d = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}, \qquad D_d = 0 \tag{15}$$

The resulting discrete-time model is:

$$\begin{aligned} \mathbf{x}_{k+1} &= A_d \mathbf{x}_k + B_d \delta_k + w_k \\ y_k &= C_d \mathbf{x}_k + v_k \end{aligned} \tag{16}$$

Problem 3c

To determine whether all states can be estimated, we construct the observability matrix

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \end{bmatrix} \tag{17}$$

Using the matlab script

```
%Observability
T = 169.53;
K = 0.007493;

A = [0 1 0;
     0 -1/T -K/T;
     0 0 0];

C = [1 0 0];

O = obsv(A,C);

rank_O = rank(O)
```

we get

$$\text{rank}(\mathcal{O}) = 3 \tag{18}$$

Since the rank equals the number of states, the system is fully observable. Thus, all states can be estimated using only measurements of the yaw angle.

Problem 4 - Implementation

Problem 4a

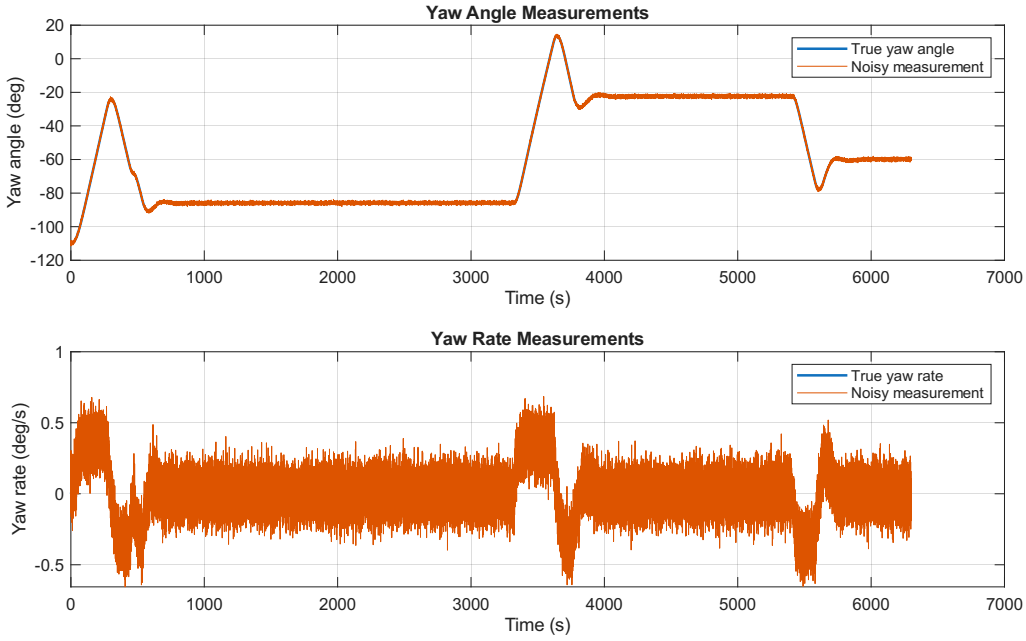


Figure 7: True and noisy measurements of the yaw angle and yaw rate, with Gaussian noise standard deviations of 0.5° and 0.1°/s, respectively.

Gaussian noise was added to the true yaw angle and yaw rate, with standard deviations of 0.5° and 0.1°/**s**, respectively. Figure 7 show that the noisy measurements fluctuate around the true states. The noise is particularly noticeable in the yaw-rate measurements due to the relatively small magnitude of the actual yaw-rate.

Problem 4b

The discrete-time Kalman filter was implemented using the first-order Nomoto model linearized at $U_{\text{ref}} = 9\text{m/s}$, with $T = -99.47$ and $K = -0.004883$. The filter uses noisy yaw-angle measurements

and the actual rudder angle to estimate the yaw angle, yaw rate and rudder bias. The results in

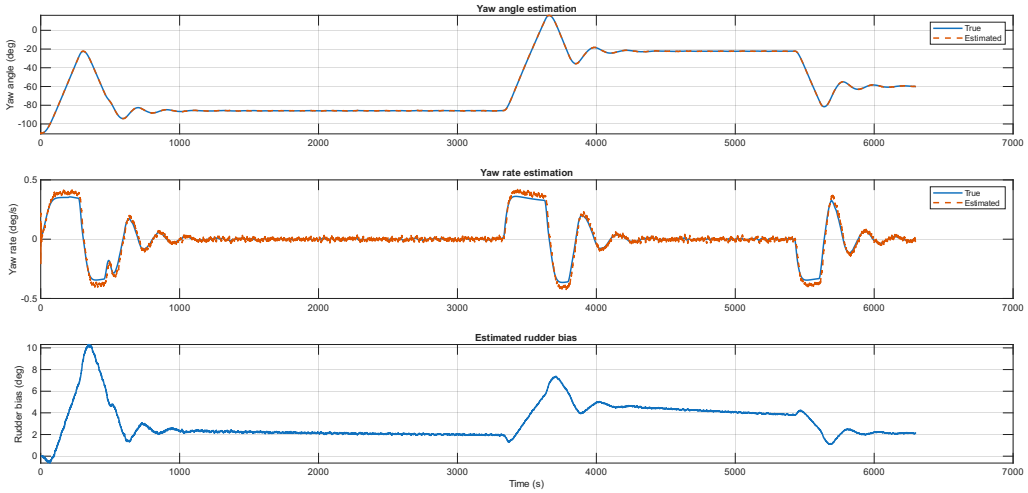


Figure 8: Kalman filter estimates of yaw angle, yaw rate and rudder bias.

Figure 8 show that the estimated yaw angle closely follows the true value throughout the simulation. The yaw rate is also accurately estimated, with small deviations during turning. Since the true rudder bias is unknown, its accuracy cannot be directly evaluated.

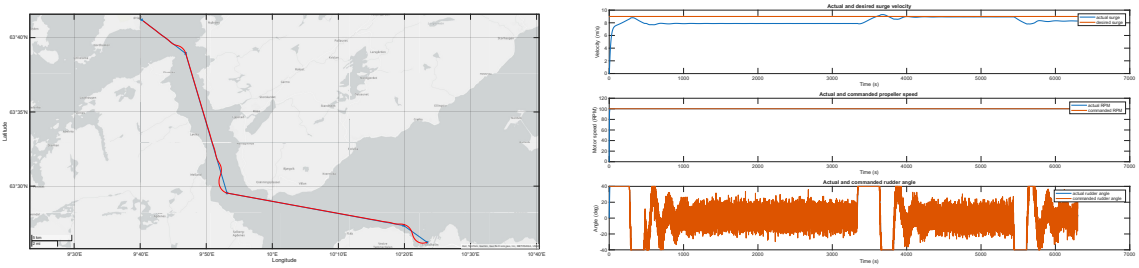
The Kalman filter was tuned by adjusting the measurement noise covariance R_d, process noise covariance Q_d and initial estimation covariance $P(0)$. The measurement covariance was set to $R_d = (1°)^2$, making the filter less sensitive to measurement noise than when using the simulated measurement variance of $(0.5°)^2$.

Small process noise covariances were selected to obtain smooth state estimation while allowing the filter to correct modeling errors. The initial covariance was chosen to represent uncertainty in the initial yaw angle, yaw rate and rudder bias, particularly since the initial yaw estimate was set to zero while the ship started at $-110°$.

The resulting estimates closely follow the true yaw angle and yaw rate, with limited noise and small deviations during turning.

Problem 4c

The path-following system was simulated using noisy yaw angle and yaw rate measurements directly as feedback to the PD heading controller.



(a) Actual and desired trajectory.

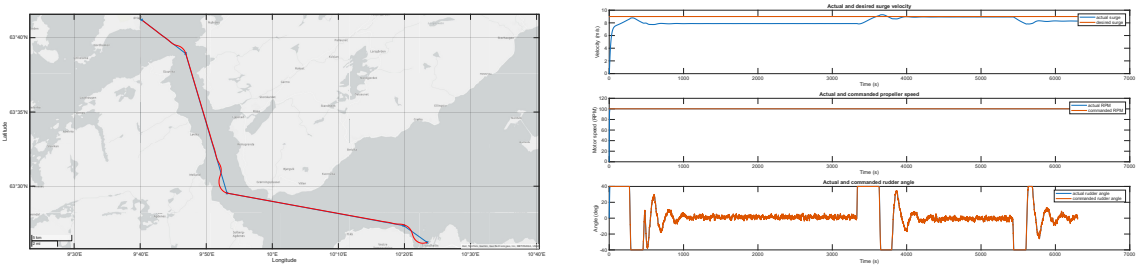
(b) Rudder angle, yaw angle, and yaw rate.

Figure 9: Path-following performance using noisy measurements as feedback.

The rusults in Figure 9 show that the ship follows the desired trajectory closely, with some overshoot during turns. The actual yaw angle and yaw rate follow their desired values reasonably well, with small deviations during maneuvers.

However the commanded rudder angle exhibits significant high-frequency oscillations due to the noisy measurements used directly as feedback. This occurs because the measurement noise is amplified by the PD controller, causing rapid changes in the commanded rudder angle.

Problem 4d



(a) Actual and desired trajectory.

(b) Rudder angle, yaw angle, and yaw rate.

Figure 10: Path-following performance using Kalman estimates as feedback.

The results in Figure 10 show that the ship follows the desired trajectory closely, with some overshoot and oscillations during turns. Compared with Problem 4c, using Kalman filter estimates as feedback significantly reduced the high-frequency oscillations in the commanded rudder angle. The yaw angle and yaw rate follow their desired values reasonably well. This demonstrates the benefit of using state estimation instead of noisy measurements directly.

Problem 5 - Navigation Systems

Problem 5a

The main intention with wave filtering in model-based navigation systems is to reduce wear and tear on the steering machine. In motion control systems, environmental forces like waves are consider disturbances and are conceptually divided into low-frequency and wave-frequency components. Only the slowly varying forces should enter the feedback loop and be counteracted for by steering and propulsion. The control systems should not attempt to counteract the wave-frequency part, and allowing the wave-frequency part to enter the loop might lead to unnecessary use of actuators, reduced tracking performance and higher fuel consumption.

Problem 5b

Model-based navigation systems uses physical, dynamic or kinematic models of the vehicle to estimate behavior and biases. This requires extensive and precise knowledge of the system, but allows for accurate estimations of disturbances and biases if the model is sufficiently accurate.

Inertial navigation systems, we use an IMU unit consisting of accelerometers and gyroscopes to calculate position, velocity and attitude by integrating the measurements received from the sensors. This does not require as extensive system knowledge. However, due to the integration (especially in the position, where acceleration is integrated twice), it is vulnerable to drift where the bias accumulates over time, and often needs to be pared with a different system (like GNSS) to correct said drift.

Problem 6 - From Model-Based KF to Inertial State Estimation with ESKF

Problem 6a

State vector and system model

The model based KF estimates the full states in the state vector, where the state vector contains system states like position, velocity or orientation. The ESKF focuses on the error state, and estimates the errors in the system states rather than the state itself. The ESKF distinguishes between the error state and the nominal state. The nominal state contains teh estimated system states like position,velocity and orientation. The error state vector consists of error states, like position, velocity and orientation error.

While KF assumes a linear model based on the vessels dynamics, an ESKF can handle non-linear models by linearize the error states rather than teh full system dynamics.

Sensor inputs and measurements

In a KF, Linear acceleration and angular rate are generated using a mathematical model and used to make predictions about the states in the state vector. The measurements are then used directly to update the prediction.

The ESKF uses IMU measurements, which measures angular velocity and acceleration directly. Since these measurements are iterated to recreate the system states, the IMU is prone to bias and drift. This is usually counteracted by including slower sensors like GNSS. THE ESKF counteracts the effect of bias by estimating the bias, together with the error states. Due to the use of error states instead of full states, the ESKF are relative computationally efficient.

Process and measurement noise

In a model based KF, process noise represents uncertainty in the dynamics model, while measurement noise represents the uncertainty in all the sensors used to update the state prediction. The KF uses the kinematic model, the measurements and the process and countermeasure to predict the mean and covariance of the full states.

In a ESKF the process noise represents uncertainty in the received IMU measurements and the INS model, while the measurement noise represents the uncertainty in the slower aiding sensors like the GNSS.

The ESKF uses the IMU, the other sensors and the measurement and process noise to predict the covariance and mean of the error states. Since the ESKF propagates the nominal state, not the error the ESKF performs an "inject and reset" step. This is where the error estimates are used to update the nominal state. After the error estimates are injected into the nominal states, the error state mean is set to zero until the next measurement update. Afterwards the error state covariance is updated to match the change in the nominal state (the reset).[4]

Problem 6b

The inertial ESKF was implemented using the `ins_euler` function, with IMU measurements at 10 Hz and GNSS position measurements 5 Hz. The estimated states closely follow the true ship states, with RMSE values of 0.0118 m and 0.0116 m for north and east position, respectively, and 0.2433° for yaw angle. The estimation errors remain small throughout the simulation, indicating accurate and stable position and heading estimation.

Problem 6c

The autopilot was simulated using the ESKF state estimates as feedback to the heading controller.

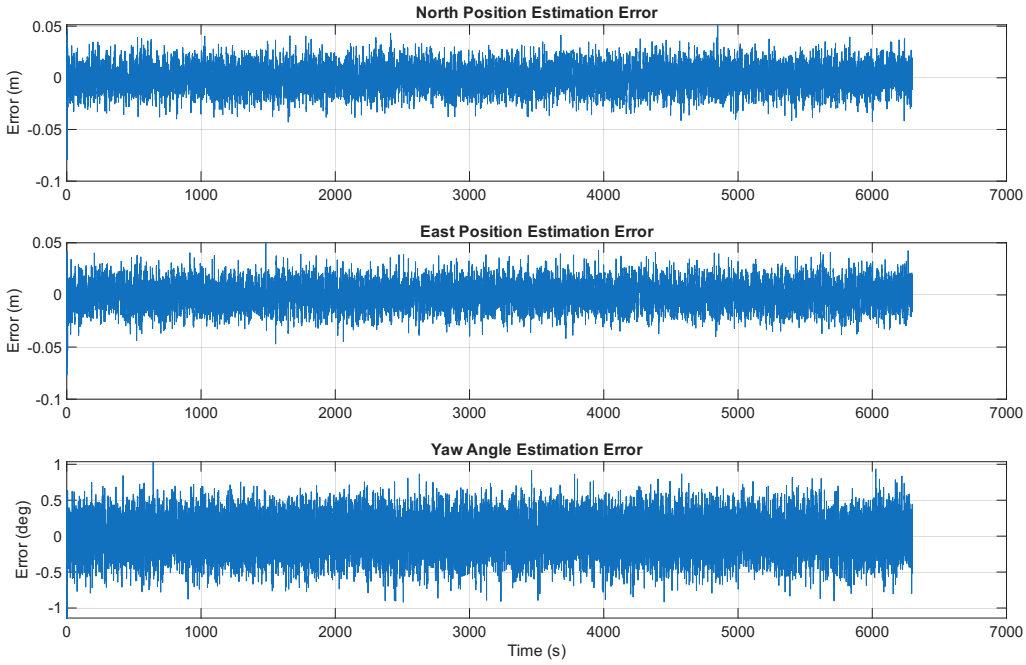
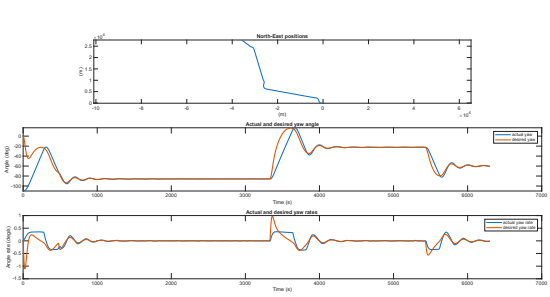
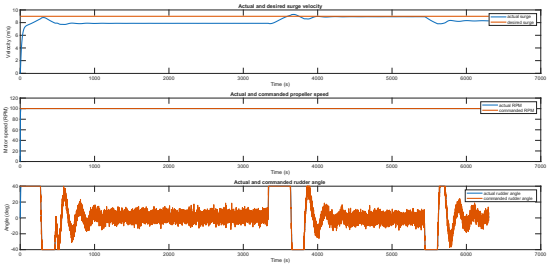


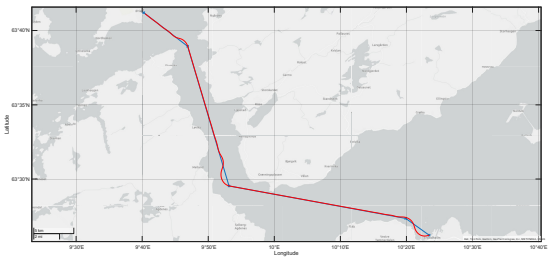
Figure 11: ESKF estimation errors for north position, east position, and yaw angle over time.



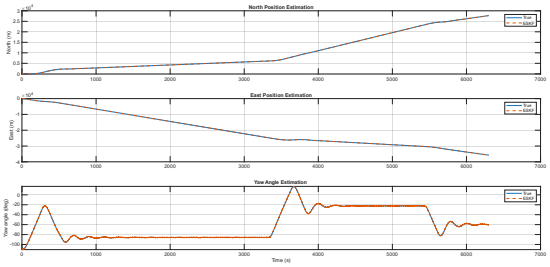
(a) Position, yaw angle and yaw rate.



(b) Surge velocity, propeller speed and rudder angle.



(c) Ship trajectory.



(d) ESKF position and yaw angle estimates.

Figure 12: Autopilot performance using ESKF state estimation.

The results in Figure 12 show that the actual yaw angle follows closely, with some overshoot and oscillations during turns. The ESKF provides accurate state estimates, as the estimated yaw angle closely matches the actual yaw angle throughout the simulation.

Problem 6d

The simulation results show that the ESKF provides accurate estimations of the ship's position and yaw angle, closely matching the true values. The autopilot remains stable and follows the desired trajectory. However, the commanded rudder angle exhibits high-frequency oscillations and reaches its saturation limits during larger maneuvers.

The promising results are to some degree explained by the fact that the filters are tested in a simulation. Even with the modeling of wind and current, the simulation represents ideal and predictable conditions. A real scenario would likely contain a multitude of complex and unpredictable disturbance that would be nearly impossible to take into account when performing a simulation. This might for example affect the real process and measurement noise. If the real noise parameters differ from the ones simulated, the filter would likely perform worse outside the simulation, and might even become inconsistent[4]. It is also important to remember that a simulation is performed in a void, with no other vehicles, obstacle or other unforeseen events. Introducing said obstacles could result in more frequent and likely more aggressive maneuvers. This is significant because the system general differs the most from the references when a turning maneuvers is performed, with larger deviations and more recovery time for more aggressive movements. Introducing several more maneuvers might cause a filter tuned in a simulation to struggle to keep up with the sudden and unpredictable state changes.

Due to the aforementioned differences between the simulation scenario and the real-world scenario I would not necessarily expect the same relative performance for a real-world implementation. However, it is worth nothing that the filter will not necessarily diverge in the real-life implementation, but might perform slightly worse that the simulated one.

References

[1] T. Fossen, *Handbook of Marine Craft Hydrodynamics and Motion Control*. John Wiley & Sons, 2011.

[2] T. I. Fossen, "Ttk4190 assignment 2, part 3," Assignment description, NTNU, 2026, accessed: 6 November 2026. [Online]. Available: https://canvas.ntnu.no/

[3] T. I. Fossen and A. P. Aguiar, "A uniform semiglobal exponential stable adaptive line-of-sight (alos) guidance law for 3-d path following," *Automatica*, vol. 163, p. 111556, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0005109824000487

[4] E. Brekke, *Fundamentals of Sensor Fusion*, 6th ed. unpublished manuscrip, 2026.